Compendio Total de Códigos en C++

Nombre: Campos Salgado Diego Campos

Rodríguez Zuñiga Owen Alexander

1. Hola mundo.cpp

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hola mundo" << endl;
    return 0;
}
```

Explicación: Es el programa más básico en C++. Utiliza la biblioteca <iostream> para la entrada y salida de datos. La función principal main imprime en la consola el texto "Hola mundo" mediante el objeto cout y el operador de inserción <<, terminando con un salto de línea (endl).

2. Calculadora basica.cpp

```
#include <iostream>
using namespace std;

int main() {
    double a, b;

    cout << "Ingresa dos numeros: ";
    cin >> a >> b;

    cout << "Suma: " << a + b << endl;
    cout << "Resta: " << a - b << endl;
    cout << "Multiplicacion: " << a * b << endl;

    if (b != 0)
        cout << "Division: " << a / b << endl;
    else
        cout << "No se puede dividir entre cero" << endl;
```

```
    return 0;
}
```

Explicación: Solicita al usuario ingresar dos números de tipo double. Posteriormente, realiza y muestra las operaciones aritméticas elementales: suma, resta, multiplicación y división. Incluye una estructura condicional (if-else) para validar que el divisor no sea cero, previniendo así un error crítico de ejecución.

3. Calculadora con operaciones recursivas.cpp

```
#include <iostream>
using namespace std;

int multiplicar(int a, int b) {
    if (b == 0)
        return 0;

    if (b < 0)
        return -multiplicar(a, -b);

    return a + multiplicar(a, b - 1);
}

int potencia(int base, int exponente) {
    if (exponente == 0)
        return 1;

    return multiplicar(base, potencia(base, exponente - 1));
}

int dividir(int a, int b) {
    if (b == 0)
        return 0;

    if (a < b)
        return 0;

    return 1 + dividir(a - b, b);
}

int main() {
    int a, b;

    cout << "Ingresa dos numeros positivos: ";
    cin >> a >> b;
```

```
    cout << "Multiplicacion: " << multiplicar(a, b) << endl;
    cout << "Potencia: " << potencia(a, b) << endl;

    if (b != 0)
        cout << "Division entera: " << dividir(a, b) << endl;
    else
        cout << "No se puede dividir entre cero" << endl;

    return 0;
}
```

Explicación: Implementa operaciones matemáticas avanzadas utilizando la recursividad (funciones que se llaman a sí mismas). La función multiplicar simula la multiplicación mediante sumas sucesivas, la función potencia utiliza multiplicaciones recursivas basadas en la función anterior, y la función dividir calcula la división entera restando de forma sucesiva el divisor.

4. Crear un tipo de dato Auto y Persona.cpp

```
#include <iostream>
#include <string>
using namespace std;

struct Auto {
    string marca;
    double precio;
    int anio;
};

struct Persona {
    string nombre;
    string ap;
    string am;
    string genero;
    int edad;
};

int main() {
    Auto a;
    Persona p;

    cout << "Datos del auto" << endl;
    cout << "Marca: ";
    getline(cin, a.marca);
    cout << "Precio: ";
    cin >> a.precio;
    cout << "Anio: ";
    cin >> a.anio;
```

```
    cin.ignore();

    cout << "\nDatos de la persona" << endl;
    cout << "Nombre: ";
    getline(cin, p.nombre);
    cout << "Apellido paterno: ";
    getline(cin, p.ap);
    cout << "Apellido materno: ";
    getline(cin, p.am);
    cout << "Genero: ";
    getline(cin, p.genero);
    cout << "Edad: ";
    cin >> p.edad;

    cout << "\nAuto: " << a.marca << ", $" << a.precio
         << ", " << a.anio << endl;

    cout << "Persona: " << p.nombre << " "
         << p.ap << " " << p.am << endl;
    cout << "Genero: " << p.genero
         << ", Edad: " << p.edad << endl;

    return 0;
}
```

Explicación: Introduce el uso de estructuras (struct) para crear tipos de datos personalizados. Define una estructura Auto (con marca, precio y año) y una estructura Persona (con nombre, apellidos, género y edad). En el programa principal, se instancian variables de estos tipos, se leen los datos usando cin y getline, y finalmente se imprimen en pantalla.

5. Acceder indirectamente a la lista de personas.cpp

```
#include <iostream>
#include <string>
using namespace std;

struct Persona {
    string nombre;
    string ap;
    string am;
    string genero;
    int edad;
};

int main() {
    Persona personas[3];
```

```
    for (int i = 0; i < 3; i++) {
        cout << "\nPersona " << i + 1 << endl;
        cout << "Nombre: ";
        cin >> personas[i].nombre;
        cout << "Apellido paterno: ";
        cin >> personas[i].ap;
        cout << "Apellido materno: ";
        cin >> personas[i].am;
        cout << "Genero: ";
        cin >> personas[i].genero;
        cout << "Edad: ";
        cin >> personas[i].edad;
    }

    Persona *p = personas;

    cout << "\nCONSULTA INDIRECTA" << endl;

    for (int i = 0; i < 3; i++) {
        cout << "\nNombre: " << (p + i)->nombre << endl;
        cout << "Apellidos: " << (p + i)->ap
             << " " << (p + i)->am << endl;
        cout << "Genero: " << (p + i)->genero << endl;
        cout << "Edad: " << (p + i)->edad << endl;
    }

    return 0;
}
```

Explicación: Demuestra el uso de punteros y aritmética de direcciones para acceder a los elementos de un arreglo de estructuras sin utilizar la notación tradicional de corchetes. Declara un puntero Persona *p apuntando al inicio del arreglo personas y utiliza el operador flecha ((p + i)->campo) para la lectura e impresión indirecta de los datos.

6. Factorial recursivo.cpp

```
#include <iostream>
using namespace std;

long long factorial(int n) {
    if (n <= 1)
        return 1;

    return n * factorial(n - 1);
}
```

```
int main() {
    int n;

    cout << "Ingresa un numero entre 0 y 20: ";
    cin >> n;

    if (n < 0 || n > 20) {
        cout << "Numero fuera del rango permitido" << endl;
    } else {
        cout << "Factorial: " << factorial(n) << endl;
    }

    return 0;
}
```

Explicación: Calcula el factorial de un número entero utilizando una función recursiva. El caso base es n <= 1 (retorna 1). Incluye una validación del rango de entrada (entre 0 y 20) para prevenir desbordamientos numéricos en el tipo de dato long long.

7. Fibonacci recursivo.cpp

```
#include <iostream>
using namespace std;

int fibonacci(int n) {
    if (n <= 1)
        return n;

    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    int n;

    cout << "Cuantos numeros de Fibonacci quieres: ";
    cin >> n;

    if (n < 1 || n > 40) {
        cout << "Ingresa un numero entre 1 y 40" << endl;
        return 0;
    }

    for (int i = 0; i < n; i++) {
        cout << fibonacci(i) << " ";
    }

    cout << endl;
```

```
    return 0;
}
```

Explicación: Genera la secuencia de la serie de Fibonacci mediante una función recursiva basada en la fórmula fibonacci(n - 1) + fibonacci(n - 2). Limita el valor máximo de entrada a 40 para evitar retrasos significativos por la complejidad exponencial del algoritmo.

8. Herencia calculadora nueva.cpp

```
#include <iostream>
using namespace std;

class Calculadora {
public:
    double sumar(double a, double b) {
        return a + b;
    }

    double restar(double a, double b) {
        return a - b;
    }

    double multiplicar(double a, double b) {
        return a * b;
    }

    double dividir(double a, double b) {
        if (b == 0) {
            cout << "Error: division entre cero" << endl;
            return 0;
        }
        return a / b;
    }
};

class CalculadoraNueva : public Calculadora {
public:
    double potencia(double base, int exponente) {
        double resultado = 1;

        for (int i = 0; i < exponente; i++)
            resultado *= base;

        return resultado;
    }
};
```

```
int main() {
    CalculadoraNueva c;

    cout << "Suma: " << c.sumar(5, 3) << endl;
    cout << "Resta: " << c.restar(5, 3) << endl;
    cout << "Multiplicacion: " << c.multiplicar(5, 3) << endl;
    cout << "Division: " << c.dividir(6, 3) << endl;
    cout << "Potencia: " << c.potencia(2, 3) << endl;

    return 0;
}
```

Explicación: Muestra los conceptos clave de la Programación Orientada a Objetos (POO). Define una clase base Calculadora y una clase derivada CalculadoraNueva que hereda sus métodos públicos y añade la capacidad de calcular potencias mediante iteración.

9. Herencia y sobreescritura.cpp

```
#include <iostream>
using namespace std;

class Calculadora {
public:
    virtual int multiplicar(int a, int b) {
        return a * b;
    }
};

class CalculadoraNueva : public Calculadora {
public:
    // Sobreescritura: multiplicacion con sumas sucesivas
    int multiplicar(int a, int b) override {
        int resultado = 0;
        int signo = 1;

        if (a < 0) {
            a = -a;
            signo = -signo;
        }
        if (b < 0) {
            b = -b;
            signo = -signo;
        }

        for (int i = 0; i < b; i++)
            resultado += a;
```

```
        return resultado * signo;
    }

    // Potencia con multiplicaciones sucesivas
    int potencia(int base, int exponente) {
        int resultado = 1;

        for (int i = 0; i < exponente; i++)
            resultado = multiplicar(resultado, base);

        return resultado;
    }

    // Division entera con restas sucesivas
    int dividir(int a, int b) {
        if (b == 0) {
            cout << "No se puede dividir entre cero" << endl;
            return 0;
        }

        int signo = 1;
        if (a < 0) {
            a = -a;
            signo = -signo;
        }
        if (b < 0) {
            b = -b;
            signo = -signo;
        }

        int cociente = 0;

        while (a >= b) {
            a -= b;
            cociente++;
        }

        return cociente * signo;
    }
};

int main() {
    CalculadoraNueva c;

    cout << "Multiplicacion: " << c.multiplicar(4, 3) << endl;
    cout << "Potencia: " << c.potencia(2, 4) << endl;
    cout << "Division entera: " << c.dividir(10, 3) << endl;

    return 0;
```

```
}
```

Explicación: Profundiza en la POO mediante herencia y sobreescritura de métodos (override). Redefine el método virtual multiplicar para realizar la multiplicación mediante sumas sucesivas e implementa métodos propios para potencia y división por restas sucesivas reconociendo signos.

10. Identificar tamaños de los tipos de datos.cpp

```
#include <iostream>
#include <string>
using namespace std;

struct Auto {
    string marca;
    double precio;
    int anio;
};

class Persona {
public:
    string nombre;
    string ap;
    string am;
    string genero;
    int edad;
};

int main() {
    cout << "Tipos basicos:" << endl;
    cout << "char: " << sizeof(char) << " bytes" << endl;
    cout << "int: " << sizeof(int) << " bytes" << endl;
    cout << "float: " << sizeof(float) << " bytes" << endl;
    cout << "double: " << sizeof(double) << " bytes" << endl;
    cout << "bool: " << sizeof(bool) << " bytes" << endl;

    cout << "\nTipos creados:" << endl;
    cout << "Auto: " << sizeof(Auto) << " bytes" << endl;
    cout << "Persona: " << sizeof(Persona) << " bytes" << endl;
    cout << "string: " << sizeof(string) << " bytes" << endl;

    return 0;
}
```

Explicación: Utiliza el operador sizeof() para determinar la ocupación en memoria RAM de los tipos primitivos de C++ (char, int, float, double, bool) y de tipos definidos por el usuario (estructuras, clases y std::string).

11. Operaciones con matrices A y B.cpp

```
#include <iostream>
using namespace std;

int main() {
    int A[2][2], B[2][2], C[2][2];
    int constante;

    cout << "Ingresa la matriz A (2x2):" << endl;
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            cin >> A[i][j];
        }
    }

    cout << "Ingresa la matriz B (2x2):" << endl;
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            cin >> B[i][j];
        }
    }

    cout << "Ingresa la constante: ";
    cin >> constante;

    cout << "\nA * constante:" << endl;
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            cout << A[i][j] * constante << "\t";
        }
        cout << endl;
    }

    // Multiplicacion de matrices A por B
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            C[i][j] = 0;

            for (int k = 0; k < 2; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
```

```
    cout << "\nA * B:" << endl;
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            cout << C[i][j] << "\t";
        }
        cout << endl;
    }

    return 0;
}
```

Explicación: Demuestra el manejo de arreglos bidimensionales (matrices de 2x2). Solicita los elementos de las matrices A y B, y realiza dos operaciones principales: el producto escalar de una matriz por una constante y la multiplicación de matrices (filas por columnas) empleando tres ciclos for anidados.

12. Polvo de Cantor.cpp

```
#include <iostream>
#include <string>
using namespace std;

void cantor(string linea, int nivel) {
    cout << linea << endl;

    if (nivel == 0)
        return;

    int tercio = linea.size() / 3;

    if (tercio == 0)
        return;

    string izquierda = linea.substr(0, tercio);
    string centro(tercio, ' ');
    string derecha = linea.substr(2 * tercio, tercio);

    cantor(izquierda + centro + derecha, nivel - 1);
}

int main() {
    int nivel;

    cout << "Ingresa el nivel (0 a 5): ";
    cin >> nivel;
```

```
    if (nivel < 0 || nivel > 5) {
        cout << "Nivel no valido" << endl;
        return 0;
    }

    string linea(81, '*');
    cantor(linea, nivel);

    return 0;
}
```

Explicación: Implementa una representación visual del conjunto fractal conocido como Polvo de Cantor utilizando recursividad de cadenas en C++. En cada nivel, divide el segmento de texto en tres partes iguales y elimina la tercera parte central reemplazándola por espacios en blanco.

13. Programación orientada a objetos (POO).cpp

```
#include <iostream>
#include <string>
using namespace std;

class Auto {
public:
    string marca;
    double precio;
    int anio;

    void mostrar() {
        cout << marca << " $" << precio
             << " " << anio << endl;
    }
};

class Persona {
public:
    string nombre, ap, am, genero;
    int edad;

    void mostrar() {
        cout << nombre << " " << ap << " " << am << endl;
        cout << "Genero: " << genero
             << ", Edad: " << edad << endl;
    }
};
```

```
int main() {
    Auto a;
    Persona p;

    a.marca = "Nissan";
    a.precio = 250000;
    a.anio = 2022;

    p.nombre = "Juan";
    p.ap = "Perez";
    p.am = "Lopez";
    p.genero = "Masculino";
    p.edad = 18;

    a.mostrar();
    p.mostrar();

    return 0;
}
```

Explicación: Muestra los fundamentos esenciales de la Programación Orientada a Objetos en C++. Define dos clases (Auto y Persona) con atributos públicos y métodos de miembro (`mostrar()`) que encapsulan la lógica de impresión de los datos de los objetos instanciados.

14. Promedio con apuntador al arreglo.cpp

```
#include <iostream>
using namespace std;

int main() {
    double numeros[5];
    double *p = numeros;
    double suma = 0, maximo, minimo;

    for (int i = 0; i < 5; i++) {
        cout << "Ingresa un numero: ";
        cin >> *(p + i);
    }

    maximo = minimo = *p;

    for (int i = 0; i < 5; i++) {
        suma += *(p + i);

        if (*(p + i) > maximo)
            maximo = *(p + i);
```

```
        if (*(p + i) < minimo)
            minimo = *(p + i);
    }

    cout << "Suma: " << suma << endl;
    cout << "Media: " << suma / 5 << endl;
    cout << "Maximo: " << maximo << endl;
    cout << "Minimo: " << minimo << endl;

    return 0;
}
```

Explicación: Calcula la suma, el promedio, el valor máximo y el mínimo de 5 números utilizando punteros y desreferenciación (`*(p + i)`). Ilustra la relación directa entre los arreglos y los punteros en C++ al recorrer la memoria de forma secuencial.

15. Promedio con arreglo.cpp

```
#include <iostream>
using namespace std;

int main() {
    double numeros[5];
    double suma = 0, maximo, minimo;

    for (int i = 0; i < 5; i++) {
        cout << "Ingresa un numero: ";
        cin >> numeros[i];
    }

    maximo = minimo = numeros[0];

    for (int i = 0; i < 5; i++) {
        suma += numeros[i];

        if (numeros[i] > maximo)
            maximo = numeros[i];

        if (numeros[i] < minimo)
            minimo = numeros[i];
    }

    cout << "Suma: " << suma << endl;
    cout << "Media: " << suma / 5 << endl;
    cout << "Maximo: " << maximo << endl;
    cout << "Minimo: " << minimo << endl;
```

```
    return 0;
}
```

Explicación: Realiza el cálculo estadístico básico (suma, media, máximo y mínimo) sobre un conjunto de 5 números ingresados por el usuario, utilizando un arreglo unidireccional simple y acceso por índice estándar (`numeros[i]`).

16. Promedios y arreglos.cpp

```
#include <iostream>
using namespace std;

int main() {
    double n[5], suma = 0;
    double maximo, minimo;

    for (int i = 0; i < 5; i++) {
        cout << "Numero " << i + 1 << ": ";
        cin >> n[i];
        suma += n[i];
    }

    maximo = minimo = n[0];

    for (int i = 1; i < 5; i++) {
        if (n[i] > maximo) maximo = n[i];
        if (n[i] < minimo) minimo = n[i];
    }

    cout << "Suma: " << suma << endl;
    cout << "Promedio o media: " << suma / 5 << endl;
    cout << "Maximo: " << maximo << endl;
    cout << "Minimo: " << minimo << endl;

    return 0;
}
```

Explicación: Una variante optimizada del cálculo de promedios con arreglos donde la suma acumulada se realiza directamente dentro del primer ciclo de lectura de datos, optimizando el flujo del programa.

17. Sobrecarga de funciones.cpp

```
#include <iostream>
using namespace std;
```

```
int calcular() {
    return 0;
}

int calcular(int a, int b) {
    return a + b;
}

int calcular(int a, int b, int c) {
    return a + b + c;
}

int main() {
    cout << "Sin parametros: " << calcular() << endl;
    cout << "Con 2 parametros: " << calcular(5, 3) << endl;
    cout << "Con 3 parametros: " << calcular(5, 3, 2) << endl;

    return 0;
}
```

Explicación: Ilustra el concepto de sobrecarga de funciones (polimorfismo en tiempo de compilación). Define tres funciones con el mismo nombre (`calcular`) pero con diferente cantidad de parámetros, permitiendo al compilador elegir cuál ejecutar según los argumentos enviados.

18. Suma de dos numeros.cpp

```
#include <iostream>
using namespace std;

int sumar(int a, int b) {
    return a + b;
}

int main() {
    int n1, n2;

    cout << "Ingresa dos numeros: ";
    cin >> n1 >> n2;

    cout << "La suma es: " << sumar(n1, n2) << endl;

    return 0;
}
```

Explicación: Muestra la modularización básica en C++ mediante el uso de funciones definidas por el usuario. La función `sumar` recibe dos números enteros como argumentos por valor y retorna la suma de ambos.

Conclusión General

Este compendio integra la totalidad de los 18 programas desarrollados en C++, conformando una guía didáctica, completa y estructurada para el aprendizaje de la programación funcional y orientada a objetos.

A lo largo de los programas se estudian de forma gradual los conceptos fundamentales: desde la sintaxis inicial e interactividad con el usuario, pasando por el manejo de estructuras de datos lineales y bidimensionales (arreglos y matrices), el control avanzado de memoria mediante punteros y desreferenciación, hasta la abstracción mediante funciones recursivas (tales como la serie de Fibonacci, el factorial y el fractal Polvo de Cantor).

Finalmente, se consolidan las técnicas de modularidad y reusabilidad del software a través de la sobrecarga de funciones y las características clave de la Programación Orientada a Objetos (clases, objetos, métodos, herencia y sobreescritura de métodos). En conjunto, esta colección proporciona el bagaje técnico necesario para dominar tanto la lógica algorítmica como la construcción eficiente de proyectos en C++.